Single-Cycle RISC-V Processor and Calculator on FPGA

Antonio Persico

Academic Year 2025/26

1 Introduction

This report, prepared for the Advanced Logic Design course, details the design, implementation, and verification of a non-pipelined (single-cycle) RISC-V processor. The core is written in VHDL and implemented on a Digilent Nexys 4 development board. To demonstrate its functionality, the processor executes a basic calculator program, interacting with the user via memory-mapped peripherals: 16 switches, 5 push-buttons, and an 8-digit 7-segment display.

2 Architecture

2.1 Overview

The system consists of the RISC-V core, a data RAM, an MMIO decoder connecting the core to its peripherals, five button debouncers, a seven-segment display driver, and a clock generator. The core fetches instructions from an internal program ROM, while data accesses are routed through the MMIO decoder, maintaining separate instruction and data paths.

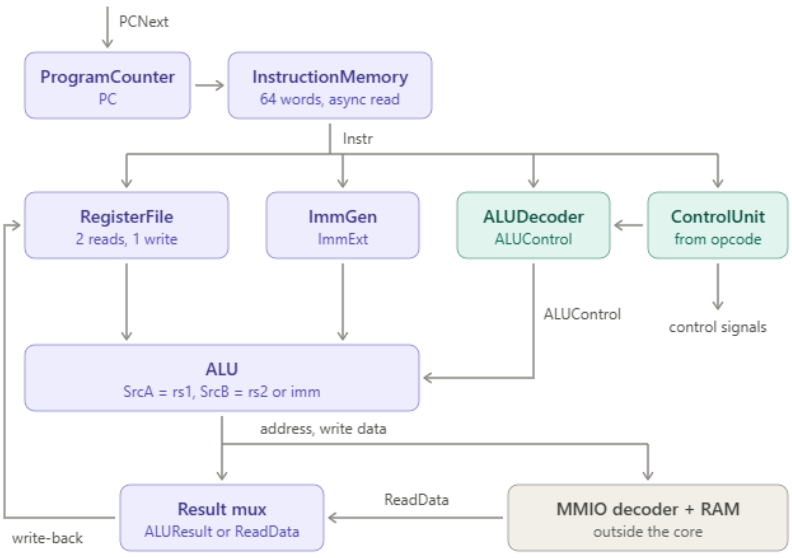


Figure 1: Block diagram of the datapath.

2.2 Instruction Set

The processor features a tailored subset of the RV32I instruction set architecture, providing the exact functionality required for the target application.

Table 1: Implemented operations.

Instruction	Format	Operation
add, sub, and, or, xor, slt	R	rd = rs1 op rs2
sll, srl	R	shift by the 5 low bits of rs2
addi, andi, ori, xori, slti	I	rd = rs1 op immediate
slli, srli	I	shift by an immediate amount
lw	I	rd = memory[rs1 + imm]
sw	S	memory[rs1 + imm] = rs2
beq	B	if rs1 = rs2 then PC = PC + imm
mret	-	return from ISR (PC = mepc)

The architecture omits certain features, specifically sra/srai, bne, blt/bge, jal/jalr, lui/auipc, sltu, along with byte and half-word memory accesses. To perform an unconditional jump, the core utilizes a beq x0, x0, offset branch, while subtracting a constant is handled by issuing an addi instruction with a negative immediate value.

2.3 Datapath

Each instruction is fully executed within a single clock cycle by progressing through the following sequential stages.

1. Fetch. The 32-bit Program Counter provides the address for the Instruction Memory. This memory is implemented as a 64-word ROM with asynchronous read access, indexed using address bits 7 down to 2.

2. Decode. The Control Unit decodes the instruction opcode, while the ALU Decoder determines the specific operation using funct3 and bit 30. Operands are supplied by the Register File, which features 32 registers, two asynchronous read ports, one synchronous write port, and a hardwired zero at x0. Additionally, the ImmGen module constructs the sign-extended immediates required for the I, S, and B instruction formats.

3. Execute. A multiplexer determines whether the second ALU operand is drawn from a register or an immediate value. The ALU executes AND, OR, ADD, XOR, SLL, SRL, SUB, or SLT operations, asserting the Zero flag if the computation yields zero, and evaluates beq instructions by performing a subtraction.

4. Memory. The ALU output serves as the data memory address. Load and store operations are routed through the MMIO decoder, which supplies the corresponding read data.

5. Write-back. A multiplexer selects either the ALU result or the retrieved memory data to be written back into the destination register.

The next Program Counter (PC) value is determined according to a strict priority scheme: interrupts, mret instructions, taken branches, and finally, sequential execution. A branch is considered taken when both the Branch control signal and the Zero flag are asserted.

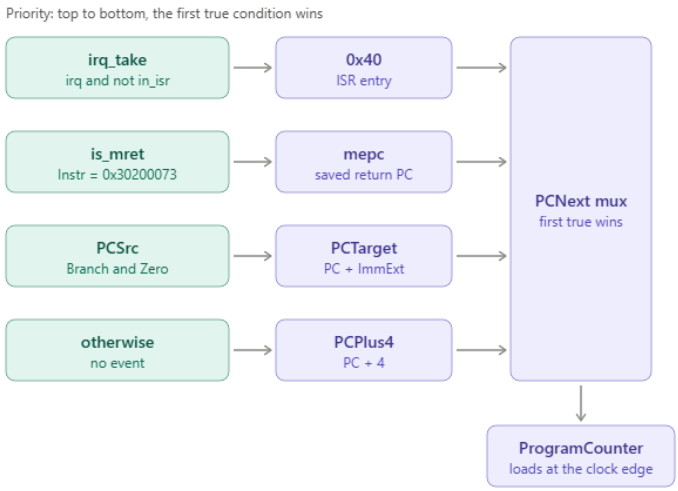


Figure 2: Next-PC selection.

2.4 Control

The control logic is organized into two hierarchical levels. The primary Control Unit generates the main control signals alongside a 2-bit ALUOp class, which the ALU Decoder subsequently refines into a precise 3-bit ALUControl signal.

Specifically, an ALUOp of 00 dictates an addition for address calculations, while 01 specifies a subtraction used for beq comparisons. For R-type instructions (ALUOp = 10), funct3 selects the base operation, with bit 30 differentiating between subtraction and addition. Conversely, for I-type instructions (ALUOp = 11), bit 30 is ignored as it forms part of the immediate value; this ensures that operations like addi with a negative immediate correctly remain additions even when bit 30 is set. The resulting 3-bit ALUControl codes are mapped as follows: 000 (AND), 001 (OR), 010 (ADD), 011 (XOR), 100 (SLL), 101 (SRL), 110 (SUB), and 111 (SLT).

Furthermore, instruction decoding is partial, evaluating only the opcode and, for ALU operations, funct3. Consequently, unsupported instructions sharing an opcode with an implemented one will alias to it, causing, for instance, bne to behave identically to beq.

2.5 Memory map and Peripherals

Peripherals are memory-mapped and accessed via standard load word (lw) and store word (sw) instructions. The MMIO decoder evaluates the target address and routes the memory access to the appropriate peripheral or memory block.

Table 2: Implemented operations.

Address	**Peripheral**
0x00 - 0xFC	Data RAM (64 words)
0x100	16 switches
0x200	Seven-segment display register
0x300	5 debounced buttons
0x400	Free-running 16-bit counter ('random' number)

RAM Write Access

The RAM write enable signal is asserted by the MMIO decoder, requiring 'MemWrite' to be high and the target address to be below '0xFF'. This address bounding is crucial because the RAM ignores upper address bits; without it, a store operation to the display at address '0x200' would inadvertently overwrite RAM word 0.

Seven-Segment Display

The 32-bit display register drives eight hexadecimal digits via time-division multiplexing controlled by a scan counter. Each digit is sequentially illuminated for approximately 2.6 ms, yielding an overall refresh rate of roughly 48 Hz.

Button Debouncing

Each button features a dedicated debouncing circuit that registers a state change only after the input remains stable for 1,000,000 clock cycles (20 ms at a 50 MHz clock frequency), resetting its internal counter upon any fluctuation. The module provides two outputs: a single-cycle pulse triggered upon a confirmed press to assert an interrupt, and a stable logic level read by the interrupt service routine to identify the active buttons.

Pseudo-Random Number Generation

A free-running hardware counter mapped at address '0x400' increments continuously on every clock cycle, wrapping around every 65,536 cycles. Sampling this counter concurrently with an asynchronous user button press yields an unpredictable value, effectively serving as a pseudo-random number source.

2.6 Interrupt Mechanism

The main program consists of an idle loop. Every confirmed button press triggers an interrupt request (IRQ), generated by logically ORing the pulse outputs from the five debouncers. When an interrupt request is acknowledged (i.e., the IRQ signal is asserted and the in-isr flag is clear):

- The Program Counter is loaded with 0x40, the base address of the Interrupt Service Routine (ISR).
- The mepc register saves the address of the interrupted instruction.

• The in-isr flag is asserted, masking any further interrupt requests while the ISR executes.

• The RegWrite and MemWrite control signals are deasserted during that clock cycle to squash the interrupted instruction, preventing any architectural state changes and ensuring it is correctly re-executed upon return.

To conclude the routine, the processor executes an mret instruction, which the decoder identifies by its complete 0x30200073 machine code. This operation seamlessly restores the Program Counter from the mepc register while deasserting the in-isr flag to allow future interrupts.

2.7 Calculator Program

The software, stored within a 64-word ROM (with the main loop at indices 0–2 and the ISR occupying indices 16–57), operates primarily on accumulator register x3, the contents of which are routed to the display. Upon execution, the ISR samples the button states into register x4, where isolated button presses yield values of 1, 2, 4, or 8. If x4 evaluates to 0, indicating the button was released, the ISR returns immediately. Otherwise, it loads the switch states into x1, extracts the operation mode into x6 (switches), and constructs a 0x3FFF bitmask to isolate the operand (switches[13:0]). Because the 0x3FFF constant exceeds the 12-bit signed immediate limit of the andi instruction, and the lui instruction is omitted from this architectural subset, the mask is dynamically generated using a logical shift followed by a subtraction. Finally, a sequence of beq branch instructions decodes the selected operation, the computed result is written to the memory-mapped display at address 0x200, and the ISR terminates.

Table 3: Button operations.

Button	**Operation (x1 = switches[13:0])**
Right	x3 = x3 AND x1
Left	x3 = x3 OR x1
Down	x3 = x3 - x1
Up	x3 = x3 + x1
Center, mode 00	x3 = x3 XOR x1
Center, mode 01	$x3 = x3 \gg x1[4:0]$
Center, mode 10	$x3 = x3 \ll x1[4:0]$
Center, mode 11	x3 = random counter

If multiple buttons are pressed simultaneously, the firmware defaults to the center-button execution path, as the combined value in register x4 will not match any of the predefined single-button encodings.

2.8 Clocking and Top Level

The development board supplies a 100 MHz reference clock on pin E3. However, because the architecture fails to achieve timing closure at this frequency (as detailed in Section 3), a Clocking Wizard IP core (MMCM) is utilized to synthesize a 50 MHz system clock that drives the entire design. The primary input clock constraints are automatically derived from the IP. Additionally, false path constraints are defined for all asynchronous inputs, namely the switches, buttons, and reset signal, as well as for the display outputs. The global system reset is mapped directly to the board's active-low CPU RESET button.

3 Implementation and performance evaluation

3.1 Methodology

The complete processor design was synthesized and implemented using the Xilinx Vivado Design Suite. System evaluation was conducted through a combination of behavioral simulations and post-implementation analyses. Specifically, the architecture was assessed based on four key performance metrics: timing characteristics, FPGA resource utilization, estimated power consumption, and the overall execution time of the calculator application.

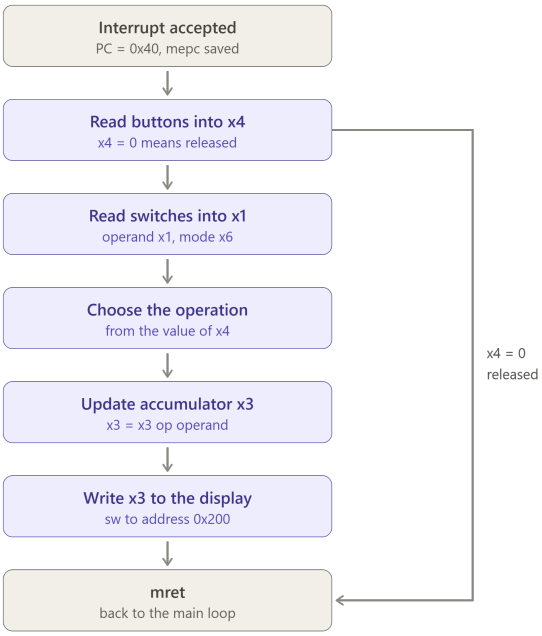


Figure 3: ISR flow.

Table 4: Timing analysis across different clock configurations.

Configuration	**Period on the core**	**WNS**	**Note**
Direct clock, 10 ns constraint	10 ns	-2.421 ns	Does not meet timing at 100 MHz
MMCM at 50 MHz, final design	20 ns	+4.510 ns	WHS +0.104 ns, WPWS +3.000 ns
MMCM at 62.5 MHz	16 ns	+1.505 ns	WHS +0.102 ns

3.2 Timing

At an operating frequency of 50 MHz, all timing constraints are successfully met, with no failing endpoints and no unconstrained paths. Based on the Worst Negative Slack (WNS) of this implementation, the estimated maximum frequency (F_{max}) can be calculated as follows:

$$F_{max} = \frac{1}{20\,\text{ns} - 4.510\,\text{ns}} \approx 64.6\,\text{MHz} \tag{1}$$

To validate this timing margin, a confirmation run was executed at 62.5 MHz (with a clock period of 16 ns). This configuration also met all timing requirements, yielding a positive WNS of +1.505 ns and thus confirming that the design safely operates at 62.5 MHz. Extrapolating from the slack of this second run provides a slightly higher theoretical maximum frequency:

$$F_{max} = \frac{1}{16\,\text{ns} - 1.505\,\text{ns}} \approx 69.0\,\text{MHz} \tag{2}$$

Critical path

The critical path of the processor originates at the Program Counter register and propagates through the Instruction ROM, the Register File read port (implemented as distributed RAM), the Arithmetic Logic Unit (ALU), the Zero flag evaluation logic, and the next-PC multiplexer, before terminating back at the PC. This path exhibits a total propagation delay of 15.109 ns across 14 logic levels, with the delay distribution comprising approximately 25% logic and 75% routing.

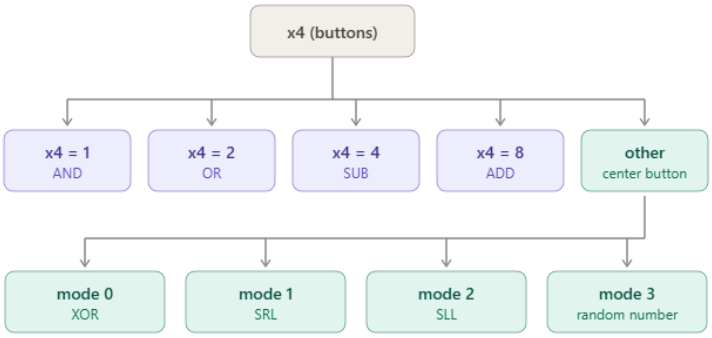


Figure 4: Operation dispatch.

3.3 Resource utilization

Table 5: FPGA resource utilization.

Resource	Used	Utilization
LUT (logic)	592	0.93%
LUT used as memory (LUTRAM)	76	0.40%
Flip-flops	200	0.16%
CARRY4	66	–
BRAM / DSP	0 / 0	0%

The Instruction ROM, Data RAM, and Register File are implemented using standard logic and distributed RAM (LUTRAM) rather than Block RAM (BRAM). This architectural choice is dictated by the single-cycle datapath, which inherently requires asynchronous read operations, whereas FPGA BRAM macros exclusively support synchronous reads. Consequently, the total LUTRAM utilization is attributed entirely to the Register File (44 LUTs) and the Data RAM (32 LUTs). Furthermore, a substantial portion of the sequential logic, approximately 115 flip-flops, is allocated to the five input debouncer modules.

3.4 Power

Table 6: Power consumption estimates reported by Vivado.

Component	Power
Total on-chip	0.219 W
Device static	0.097 W
Dynamic	0.121 W
of which MMCM	0.106 W
of which Logic, signals, clocks, I/O	$\approx$ 0.015 W

It should be noted that these power figures are derived from a vectorless estimation based on default switching activity rates, resulting in a low overall confidence level. The analysis reveals that the clock generation circuitry (MMCM) dominates the dynamic profile, accounting for nearly half of the total on-chip power. In contrast, the dynamic power dissipated by the processor core logic remains marginal. Under these assumed operating conditions, the estimated junction temperature is 26.0 °C.

3.5 Execution time

Table 7: Instruction count and execution time for calculator operations at 50 MHz.

Operation	**Instructions**	**Time at 50 MHz**
AND / OR / SUB / ADD	15 / 17 / 19 / 21	300 / 340 / 380 / 420 ns
Center: XOR / SRL / SLL / random	23 / 25 / 27 / 28	460 / 500 / 540 / 560 ns
Release (no button pressed)	3	60 ns

Given the single-cycle architecture with a Cycles Per Instruction (CPI) of 1 and a clock period of 20 ns, each instruction requires exactly 20 ns to execute. The total number of instructions processed by the Interrupt Service Routine (ISR), including the final `mret` instruction, varies depending on the specific operation requested. Consequently, any operation is fully serviced in under 0.6 μs following the interrupt trigger. This processing time is entirely negligible when compared to the 20 ms mechanical debouncing interval, which ultimately dictates the user-perceived responsiveness of the system. Finally, operating at 50 MHz with an execution rate of one instruction per clock cycle, the processor achieves a peak throughput of 50 MIPS (Million Instructions Per Second).

4 Verification

4.1 Strategy

The verification process adopts a hierarchical strategy. At the module level, each component is validated using a self-checking VHDL testbench that compares the module's outputs against expected values or a reference model. These testbenches automatically report a `PASS` or `FAIL` status for each assertion, concluding with a final summary of the simulation results. At the system level, a core testbench executes the real application program alongside simulated peripherals. Finally, the complete system was synthesized and extensively tested on the physical FPGA board.

4.2 Testbenches

Table 8 summarizes the individual testbenches developed to verify the datapath, control, and peripheral modules of the processor.

4.3 Core and Interrupt Tests

The `tb_Core` testbench evaluates the processor's interrupt handling by simulating the debouncer output (a single-cycle pulse), applying various switch and button stimuli, and comparing the value written to the 7-segment display against a reference model of the Interrupt Service Routine (ISR). The verification covers the following scenarios:

- System reset and main program execution (verifying the display outputs 0).
- Arithmetic and logic operations (ADD, OR, AND, SUB), including negative results and proper masking of the two mode-selection bits.
- Center button actions, including XOR, logical SRL, SLL (testing shift amounts of 0, 31, and 33), and random value generation.
- Simultaneous presses of multiple buttons.
- Button release events (verifying no spurious display writes occur), followed by standard operations.
- Occurrence of a secondary interrupt during an active ISR, verifying that nested or overlapping interrupts are correctly ignored.

Table 8: Summary of module-level testbenches and verified features.

Testbench	**Verified Features**
`tb_ALU`	-All 8 arithmetic/logic operations and the Zero flag. -Corner cases including overflow, subtraction below zero, and shift amounts of 0, 31, 32, and 33 positions. -Signed comparisons.
`tb_ImmGen`	-Proper decoding of I-, S-, and B-type formats using positive, negative, and boundary values. -Behavior with unused `ImmSrc` selectors.
`tb_Control`	-`ControlUnit` outputs for all valid and unused opcodes. -Complete `ALUDecoder` truth table; -End-to-end routing of 20 complete instructions through the `ControlUnit` and `ALUDecoder`.
`tb_RegFile`	-Hardwiring of register `x0` to zero. -Masking writes when Write Enable is low. -Behavior during simultaneous dual-port reads. -Asynchronous read immediately reflecting a synchronous write. -Exhaustive write/read sweep across all 31 general-purpose registers.
`tb_MMIO`	-Memory-mapped reads from switches, buttons, the random number counter, and RAM. -Write-only constraint enforcement on the display at address `0x200`. -Ignored writes to read-only peripheral addresses. -Restriction of RAM Write Enable to addresses below `0xFF`. -Reset behavior.
`tb_Debouncer`	-Generation of a single, one-cycle pulse following a stable button press. -Rejection of mechanical bounces and ultra-short presses. -Prevention of pulses upon button release. -Correct handling of two consecutive presses. -Reset behavior.
`tb_Core`	-Integration of the core with the populated ROM and simulated peripheral responses (see Section 4.3).

- An interrupt request arriving in the exact same clock cycle as a memory store instruction in the main program, ensuring the store is correctly cancelled and the ISR is executed once.
- Assertion of the reset signal midway through the ISR, followed by a new interrupt request.

4.4 On-Board Testing

Following successful behavioral simulation, the complete system, including the custom button wiring, debouncers, interrupt logic, and the display driver, was synthesized, implemented, and loaded onto the Digilent Nexys 4 board. The hardware was ultimately validated through multiple hands-on experiments to confirm the accurate operation of the memory-mapped calculator application in a real-world environment.

5 Conclusions and Future Work

Although the implementation of the single-cycle RISC-V processor on the FPGA successfully yielded a functional calculator with memory-mapped peripherals, the current design presents several architectural limitations. Addressing these constraints offers clear pathways for future development.

Limitations of the Design

- **Restricted RV32I Subset:** The core lacks support for several instructions, including `sra`, `bne`, `blt`, `jal`, `lui`, and `sltu`, as well as byte and half-word memory accesses. Additionally, `srl` is currently implemented strictly as a logical shift.
- **Partial Instruction Decoding:** Unsupported instructions that share an opcode with implemented ones are not trapped; instead, they execute erroneously as their implemented counterparts.
- **Interrupt and Context Management:** The Interrupt Service Routine (ISR) does not perform register context saving. Furthermore, any secondary interrupt triggered during an active ISR is lost. While the practical impact is negligible, human inputs are separated by tens of milliseconds, whereas the ISR completes in under 1 µs, it remains a structural flaw.
- **Memory Mapping Inconsistencies:** The display register is write-only, and accesses to unmapped memory addresses incorrectly default to a RAM read operation.
- **Input Synchronization and Reset Anomalies:** Asserting the reset signal while simultaneously holding a push-button triggers a spurious interrupt after the debouncing period. Furthermore, button inputs lack a standard two-stage flip-flop synchronizer to protect against metastability, although the implemented 20 ms stability filter effectively mitigates this risk in practice.
- **Power Estimation Accuracy:** The current power consumption figure relies on a vectorless estimation, resulting in low statistical confidence.

Possible Improvements

- **Pipelining:** Transitioning to a pipelined architecture to significantly increase the processor's maximum operating frequency.
- **ISA Expansion:** Implementing the missing instructions (e.g., `sra`, `bne`, `jal`) to achieve fuller compliance with the RV32I standard.
- **Robust Input and Interrupt Handling:** Upgrading the interrupt controller by introducing a pending-interrupt flag, hardware-level context saving during the ISR, and a dedicated dual flip-flop synchronizer on all asynchronous inputs.